

🕒 パスワードチャレンジ WS

JS で動きのあるゲームを作ろう！

前回の p2hacks 実績ページ WS の続き

今日作るもの

ストップウォッチで測りながら、**7つの条件**を全部クリアするパスワードを作るゲーム

ゲームの流れ

1. **スタートボタン** を押す → タイマー開始
2. パスワード入力欄に入力する
3. **1文字入力するたびに**条件を判定
4. 条件クリアで **✖** → **✓** に変化
5. **全条件クリア** → 成功画面表示！

使うJSの技術（今日学ぶこと）

- `input` イベント（リアルタイム判定）
- 正規表現（ `/[A-Z]/.test()` ）
- `setInterval` / `clearInterval`
- `classList.add()` / `remove()`

7つの条件リスト

#	条件	判定の仕組み
1	8文字以上	<code>value.length ≥ 8</code>
2	大文字（A～Z）を含む	<code>/[A-Z]/.test(val)</code>
3	数字（0～9）を含む	<code>/[0-9]/.test(val)</code>
4	記号（!@#\$等）を含む	<code>/[!@#\$%...]/.test(val)</code>
5	¥（円記号）を含む	<code>val.includes('¥')</code>
6	全角「A」と半角「I」が連続	<code>/Aℓ/.test(val)</code>
7	 （寿司絵文字）を含む	<code>val.includes('🍣')</code>

- 条件は **1文字入力するたびにリアルタイムで判定** される！

【ステップ1】 要素を取得する

今日も `document.querySelector()` で始まる。
取得する要素が**前回より多い**ので、まとめて確認しよう。

```
const passwordInput = document.querySelector('#password-input'); // 入力欄
const startBtn      = document.querySelector('#start-btn');     // スタートボタン
const timerDisplay   = document.querySelector('#timer');         // タイマー表示

// ここから下はまだ要素が取得できていません！
const celebration    = null;
const finalTimeEl    = null;
const retryBtn       = null;
```

- 前回と同じ書き方！ **#** をつけてIDで指定する



【ワーク】残りの要素を取得してみよう！

`main.js` の【ステップ1】にある以下の3つの変数に、正しい要素を代入してください。

- `celebration` : クリア画面の枠 (`#celebration`)
- `finalTimeEl` : 最終タイム表示要素 (`#final-time`)
- `retryBtn` : もう一度ボタン (`#retry-btn`)

// 解答例

```
const celebration = document.querySelector('#celebration');  
const finalTimeEl = document.querySelector('#final-time');  
const retryBtn = document.querySelector('#retry-btn');
```

⚠ 今、コンソールに赤いエラーが出ていませんか？

それは正常です！ ステップ1を完成させるまで、ブラウザの開発者ツールに次のようなエラーが出ます。

```
Uncaught TypeError: Cannot read properties of null (reading 'addEventListener')
```

なぜ出るの？

- `retryBtn` がまだ `null` (何も取得できていない)
- `null.addEventListener(...)` は実行できないのでエラーになる

エラーメッセージの読み方

`○○ of null` → 「`null` に対して○○しようとした」という意味。

「変数が想定と違う中身 (`null`) になっている」ことを教えてくれるヒントとして読む練習をしよう。

- ステップ1を正しく完成させると、このエラーは自然に消えます

【ステップ2】 input イベントとは？

前回学んだのは **click** イベント（クリックで発火）。

今回は **input** イベント（入力欄の文字が変わるたびに発火）を使う。

```
// 入力欄の文字が変わるたびに実行される
passwordInput.addEventListener('input', function() {

  const value = passwordInput.value; // 今の入力内容を取得
  console.log(value);                // 入力のたびにコンソールに表示される！

});
```

イベントの種類まとめ

イベント名	発火するタイミング
click	クリックしたとき
input	テキストが変わるたびに（キー入力ごと）
keydown	キーボードのキーを押した瞬間

【ステップ2】 条件の定義（配列＋オブジェクト）

各条件を**オブジェクト**でまとめ、それを**配列**に入れて管理する。

```
const conditions = [  
  {  
    id: 'cond-length',  
    // val.length = 入力文字数  
    check: function(val) { return val.length ≥ 8; }  
  },  
  {  
    id: 'cond-uppercase',  
    // /[A-Z]/ = 「A～Zのどれか」という正規表現パターン  
    check: function(val) { return /[A-Z]/.test(val); }  
  },  
  // ... ここから下は未完成！  
];
```

- **オブジェクト** = `{}` で囲んだデータの集まり（`id` と `check` をセットで持つ）
- **配列** = `[]` で囲んだリスト（`conditions[0]` で最初の条件にアクセスできる）



【ワーク】残りの条件を書いてみよう！

`main.js` の【ステップ2】にある以下の条件判定関数を埋めてみよう。
スライドの正規表現や `.includes()` の説明がヒントです。

1. 数字（0～9）を含む (`cond-number`)

- ヒント: 正規表現で「0～9のいずれか」を表すパターンを使いましょう

2. 記号を含む (`cond-symbol`)

- ヒント: 記号のパターン `/[!@#$%^&*()\-_+=◇?]/` を使いましょう

3. ¥（円記号）を含む (`cond-yen`)

- ヒント: 正規表現ではなく `.includes()` を使いましょう

【ステップ2】 正規表現とは？

正規表現 (Regular Expression) = 文字列のパターンを表す記法。

`/.../` のスラッシュで囲んで書く。

```
/[A-Z]/.test("Hello") // → true   (H が大文字)
/[A-Z]/.test("hello") // → false  (大文字なし)

/[0-9]/.test("abc123") // → true   (1 が含まれる)
/[0-9]/.test("abcdef") // → false  (数字なし)

// includes() は正規表現なしで「そのまま」含まれるか確認できる
"hello 🍌 world".includes('🍌') // → true
"hello world".includes('¥')    // → false
```

パターンの読み方

書き方	意味
<code>[A-Z]</code>	A～Z の大文字 1文字
<code>[0-9]</code>	0～9 の数字 1文字
<code>[!@#]</code>	!、@、# のどれか

【ステップ2】 全条件を forEach でチェック

```
passwordInput.addEventListener('input', function() {  
  if (!isRunning) return; // タイマーが動いていなければスキップ  
  
  const value = passwordInput.value;  
  let allClear = true; // 全条件クリアフラグ  
  
  // conditions 配列をひとつずつ処理  
  conditions.forEach(function(condition) {  
    const element = document.querySelector('#' + condition.id);  
    const iconEl = element.querySelector('.cond-icon'); // x や ✓ の要素  
  
    if (condition.check(value)) {  
      element.classList.add('clear'); // → 緑クラスを追加  
      iconEl.textContent = '✓'; // → アイコンを変更  
    } else {  
      element.classList.remove('clear'); // → 緑クラスを削除（赤に戻る）  
      iconEl.textContent = 'x';  
      allClear = false; // 1つでも未達成ならフラグをfalseに  
    }  
  });  
  
  if (allClear) { celebrate(); } // 全クリアで演出（成功表示）へ！  
});
```

【ステップ2】 `classList` と CSS はセットで動く

`element.classList.add('clear')` は「クラスを付けるだけ」。

見た目（色が赤→緑に変わる）は CSS 側があらかじめ用意している。

CSSでの見た目の定義 (`style.css`)

```
/* 通常時（未クリア）は赤系 */
.cond-icon {
  color: #e84040;
}

/* .clear が付くと緑系に変化 */
.condition-item.clear .cond-icon {
  color: #00cc66;
}
```

JSでの操作

```
// クラスを付ける → 緑になる
element.classList.add('clear');

// クラスを外す → 赤に戻る
element.classList.remove('clear');
```

JSは「状態（クラス）」を切り替えるだけ

色や見た目は全てCSSが担当、という役割分担を意識しよう

【ステップ3】 ストップウォッチの実装

`setInterval()` = 一定時間ごとに処理を繰り返す関数

```
let startTime    = null; // 開始時刻（ミリ秒）
let timerInterval = null; // setInterval の返り値（止めるために保存）
let isRunning    = false; // 「今動いているか？」のフラグ

startBtn.addEventListener('click', function() {
  startTime = Date.now(); // 現在時刻を取得（1970年1月1日からのミリ秒）
  isRunning = true;
  startBtn.disabled = true;
  passwordInput.disabled = false;

  // ここから下（タイマーの自動更新）が未完成！
  // timerInterval = setInterval(..., 10);
});
```



【ワーク】タイマーを動かそう！

`main.js` の【ステップ3】にあるスタートボタンのクリックイベント内で、10ミリ秒ごとに時間を更新する処理を完成させましょう。

- ヒント:
 - `setInterval(関数, ミリ秒)` を使用します
 - 経過時間 `elapsed` を計算し、`timerDisplay.textContent` を `formatTime(elapsed)` で更新します
 - あとで止めるために、返り値を変数 `timerInterval` に代入してください

```
// 解答例
timerInterval = setInterval(function() {
  const elapsed = Date.now() - startTime;
  timerDisplay.textContent = formatTime(elapsed);
}, 10);
```

【ステップ3】 時間のフォーマット変換

ミリ秒の数字を "01:23.45" のような表示に変換する関数

```
function formatTime(ms) {  
  const minutes      = Math.floor(ms / 60000);           // 分  
  const seconds      = Math.floor((ms % 60000) / 1000);  // 秒  
  const centiseconds = Math.floor((ms % 1000) / 10);     // 1/100秒  
  
  // padStart(2, '0') = 1桁のとき先頭に '0' を補って2桁にする  
  // 例: 5 → "05"、12 → "12"  
  return (  
    String(minutes).padStart(2, '0')      + ':' +  
    String(seconds).padStart(2, '0')      + '.' +  
    String(centiseconds).padStart(2, '0')  
  );  
}
```

- `Math.floor()` = 小数点以下を切り捨て (例: 3.7 → 3)
- `%` (剰余演算子) = 割った余り (例: 75 % 60 = 15)

【ステップ4】 クリア演出（成功表示）

クリア（全条件を達成）したときに呼び出される `celebrate()` 関数。

```
function celebrate() {  
  // ここが未完成！  
  // タイマーを止めて、結果を表示しよう  
}
```

- クリアしたタイミングでタイマーを止めて、表示を切り替える必要があります。



【ワーク】 クリア時の処理を書こう！

`main.js` の【ステップ4】にある `celebrate()` 関数の中身を完成させてください。

1. タイマー `timerInterval` を止める（`clearInterval` を使う）
2. `isRunning` を `false` にする
3. `finalTimeEl` のテキストに、現在のタイマー表示（`timerDisplay.textContent`）の値をセットする
4. `celebration` 要素に `is-active` クラスを追加して表示する

// 解答例

```
function celebrate() {  
  clearInterval(timerInterval);  
  isRunning = false;  
  finalTimeEl.textContent = timerDisplay.textContent;  
  celebration.classList.add('is-active');  
}
```

【ステップ5】 もう一度ボタン（リセット処理）

ここは**すでに書かれているコード**。ワークはないが、自分で読んで理解しよう！

`celebrate()` で変化させたものを、ひとつずつ**スタート前の状態に戻す**のがポイント。

```
retryBtn.addEventListener('click', function() {  
  clearInterval(timerInterval); // タイマーを止める  
  timerInterval = null;  
  startTime = null;  
  isRunning = false;  
  
  timerDisplay.textContent = '00:00.00'; // 表示を0に戻す  
  timerDisplay.classList.remove('running');  
  passwordInput.value = ''; // 入力欄を空に  
  passwordInput.disabled = true; // スタート前の状態へ  
  startBtn.disabled = false;  
  
  conditions.forEach(function(condition) { // 条件アイコンを全部xに戻す  
    const element = document.querySelector('#' + condition.id);  
    element.classList.remove('clear');  
    element.querySelector('.cond-icon').textContent = 'x';  
  });  
  
  celebration.classList.remove('is-active'); // 演出画面を閉じる  
});
```

【難しめ】条件6：全角と半角の混在

全角「**A**」(U+FF21) と半角「**I**」(U+006C) が連続する

なぜ難しいの？

- 見た目は似ているが、**コンピュータ上では別の文字**
- 通常の「A」(U+0041) とは異なる
- **全角文字** = 日本語入力で大文字Aを押すと入力できる
- **半角文字** = 英語の普通のI (エル)

判定コード

```
// /A1/ の A は全角 (コピペ注意！)  
check: function(val) {  
    return /A1/.test(val);  
}
```

```
"PassA1word" → ✓ (含まれる)  
"Password"    → ✗ (ない)  
"PA1..."    → ✗ (Pは半角A)
```

まとめ：今日学んだこと

技術	使い方	用途
<code>input</code> イベント	<code>addEventListener('input', ...)</code>	文字入力ごとに判定
正規表現	<code>/[A-Z]/.test(val)</code>	パターンマッチング
<code>includes()</code>	<code>val.includes('🍕')</code>	特定の文字が含まれるか
<code>setInterval()</code>	<code>setInterval(fn, 10)</code>	一定間隔で繰り返す
<code>clearInterval()</code>	<code>clearInterval(id)</code>	繰り返しを止める
<code>classList</code>	<code>.add()</code> / <code>.remove()</code>	クラスの付け外し

次のステップ (チャレンジ)

- タイムのランキングを `localStorage` で保存してみよう！
- 条件をランダムに増やしてみよう！

 挑戦してみよう！

全条件クリアを目指せ！

パスワード例のヒント:

¥A1!S🍒ushi9 ← これで全条件クリアできる？確認してみて！